

BAB 4 Layouting dan Styling

4.1 Tujuan Pembelajaran

1. Memahami konsep layout pada React Native menggunakan Flexbox
2. Menjelaskan fungsi dan peran StyleSheet dalam pengaturan tampilan
3. Mengatur posisi, ukuran, dan jarak antar komponen secara terstruktur
4. Menerapkan styling pada komponen React Native sesuai kebutuhan antarmuka
5. Mengimplementasikan layout dan styling dalam studi kasus aplikasi sederhana

4.2 Dasar Teori

4.2.1 Properti Layout (Flexbox) pada React Native

React Native menggunakan *Flexbox* sebagai sistem layout utama untuk mengatur posisi dan ukuran komponen pada layar. Secara default, susunan komponen bersifat vertikal (*column*), berbeda dengan Flexbox pada web yang bersifat horizontal[5].

Properti utama Flexbox meliputi:

- `flex`
- `flexDirection`
- `justifyContent`
- `alignItems`
- `alignContent`

Menurut Facebook (Meta), *Flexbox* dipilih karena mampu menyediakan sistem layout yang fleksibel, responsif, dan konsisten pada berbagai ukuran layar perangkat mobile[9]. Pengaturan layout menggunakan *Flexbox* merupakan bagian yang tidak terpisahkan dari penggunaan *StyleSheet*. Seluruh properti layout seperti *flex*, *flexDirection*, *justifyContent*, *alignItems* dan *alignContent* didefinisikan di dalam objek style yang dikelola oleh *StyleSheet*. Dengan demikian, pengaturan posisi dan susunan komponen pada layar selalu dilakukan bersamaan dengan pengaturan tampilan, sehingga kode menjadi lebih terstruktur, konsisten, dan mudah dipelihara.

1. Properti flex

Properti `flex` digunakan untuk menentukan proporsi ruang yang diambil oleh suatu komponen.

```
flex: 1
```

Penjelasan:

- `flex: 1` → komponen memenuhi seluruh ruang yang tersedia
- Nilai lebih besar berarti mengambil ruang lebih banyak

Contoh penggunaan

Code	Output
<pre> 1 import { View, Text, StyleSheet } from 'react-native' 2 import React from 'react' 3 4 export default function Latihan4() { 5 return (6 <View style={styles.container}> 7 <Text>Flex 1: Initialize the entire screen</Text> 8 </View> 9) 10 } 11 12 const styles = StyleSheet.create({ 13 container:{ 14 flex: 1, 15 backgroundColor: '#ff1900', 16 } 17 }) </pre>	 Gambar 4.1 Penggunaan flex dan background color

Penjelasan:

- **flex: 1** pada *container* memberitahu React Native bahwa View ini harus mengambil seluruh ruang layar yang tersedia
- Tanpa flex: 1, container hanya akan sebesar konten di dalamnya (tidak *full screen*)



Gambar 4.2 Tampilan tanpa adanya flex: 1

Container hanya sebesar konten di dalamnya dan layar akan terlihat "menciut" di bagian atas

- Ini adalah titik referensi untuk semua *child components* di dalamnya

```

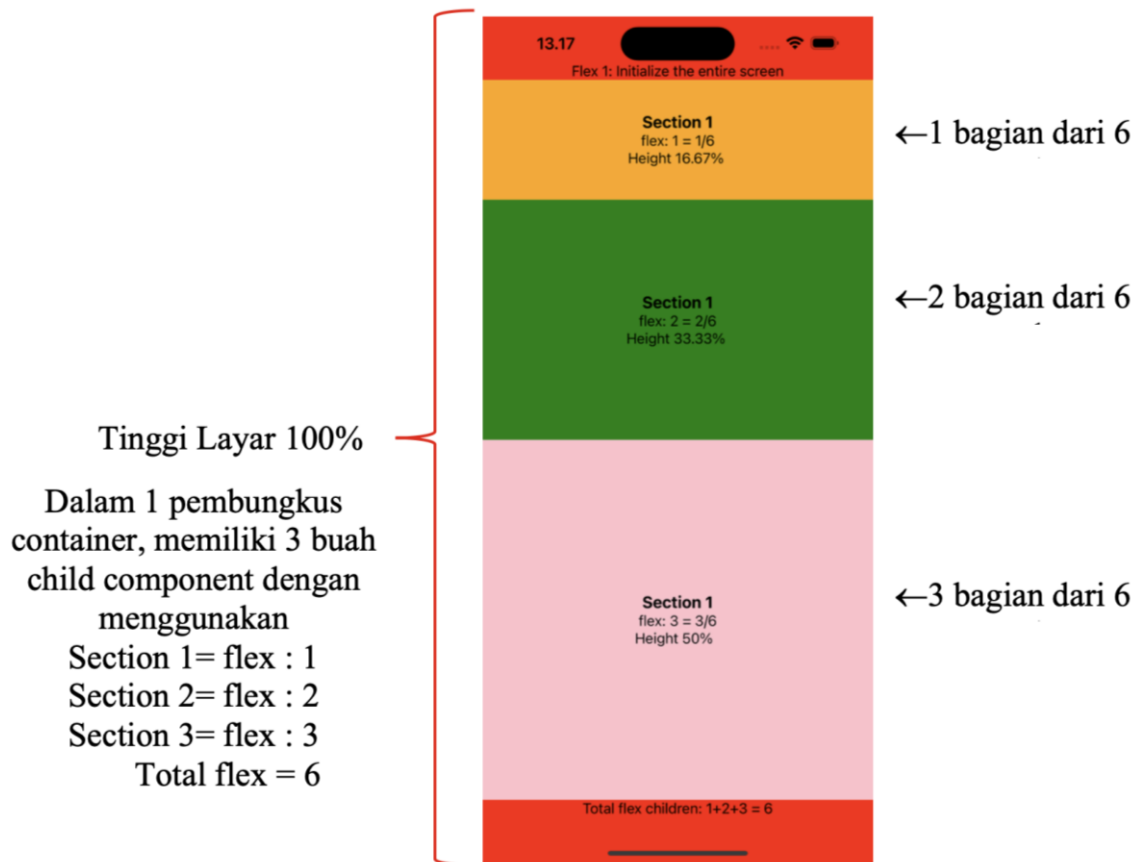
<View style={styles.container}>
  <Text>Flex 1: Initialize the entire screen</Text>
  <View style={{ flex: 1, backgroundColor: "orange" }}>
    <Text>Section 1</Text>
  </View>
  <View style={{ flex: 2, backgroundColor: "green" }}>
    <Text>Section 1</Text>
  </View>
  <View style={{ flex: 3, backgroundColor: "pink" }}>
    <Text>Section 1</Text>
  </View>
</View>

```

Penjelasan:

Tinggi Section = (flex value / total flex) × tinggi container

Section 1: $(1 / 6) \times 100\% = 16.67\%$
 Section 2: $(2 / 6) \times 100\% = 33.33\%$
 Section 3: $(3 / 6) \times 100\% = 50.00\%$

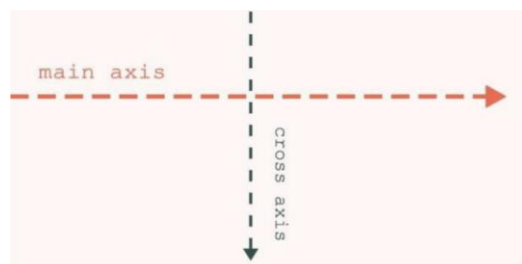


Gambar 4.3 Pembagian Section dan flex

Setiap inisialisasi pembungkus utama (*screen wrapper*) pada React Native **wajib** menggunakan Flexbox dengan **nilai flex: 1**, karena pengaturan ini memastikan tampilan dapat mengisi seluruh layar perangkat secara proporsional dan menjadi dasar utama dalam pengelolaan layout yang konsisten dan responsif.

2. Properti flexDirection

FlexDirection mengatur arah Main Axis. Dimana nilai default dari FlexDirection ialah column, yang artinya Cross Axis akan berada di sumbu vertikal, bergerak dari atas ke bawah.



Gambar 4.4 Arah Layout Flex Direction

Arah layout menentukan bagaimana komponen anak (children) dan teks disusun di dalam suatu hierarki tampilan. Pengaturan ini juga memengaruhi makna posisi start dan end pada sebuah komponen. Secara bawaan, React Native menggunakan arah tata letak *Left-to-Right* (LTR), di mana posisi start merujuk ke sisi kiri dan end merujuk ke sisi kanan layar. Berikut adalah properti yang dapat digunakan untuk mengatur arah layout, seperti dibawah ini:

`flexDirection:'column'`

Pada tabel dibawah ini merupakan value dari properti apa saja yang dapat digunakan untuk mengatur arah layout:

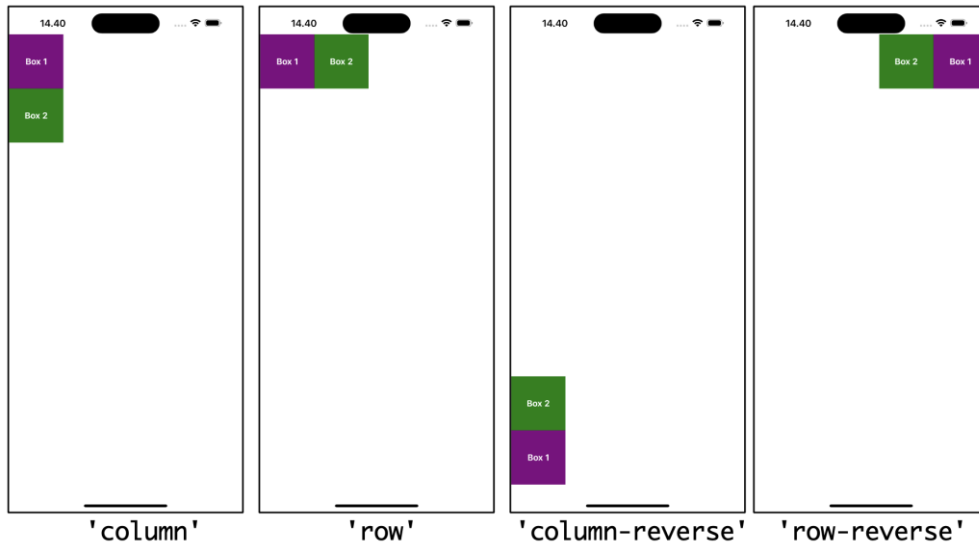
Value	Deskripsi
column	Mengatur posisi node-node berada dari atas ke bawah. Jika pembungkus diaktifkan, baris berikutnya akan dimulai di sebelah kanan item pertama di bagian atas area.
row	Mengatur posisi node-node dari kiri ke kanan. Jika pembungkus diaktifkan, baris berikutnya akan dimulai di bawah item pertama di sebelah kiri wadah.
column-reverse	Mengatur posisi node-node dari bawah ke atas. Jika pembungkus diaktifkan, baris berikutnya akan dimulai di sebelah kanan item pertama di bagian bawah wadah.
row-reverse	Mengatur posisi node-node dari kanan ke kiri. Jika pembungkus diaktifkan, baris berikutnya akan dimulai di bawah item pertama di sebelah kanan wadah.

Tabel 4.1 List Value Properti pada Flex-Direction

Implementasi arah layout dengan FlexDirection:

```

1  export default function Latihan4() {
2    return (
3      <View style={{ flex: 1, flexDirection: "column" }}>
4        <View style={{ backgroundColor: "purple" }}>
5          <Text>Box 1</Text>
6        </View>
7        <View style={{ backgroundColor: "green" }}>
8          <Text style={styles.text}>Box 2</Text>
9        </View>
10     </View>
11   );
12 }
```



Gambar 4.5 Tampilan Objex dengan Flex-Direction

3. Properti justifyContent

justifyContent digunakan untuk mengatur bagaimana komponen anak (*children*) disejajarkan sepanjang sumbu utama (*main axis*) dari sebuah kontainer. Properti ini sangat bergantung pada nilai *flexDirection* yang digunakan.

Sebagai contoh, ketika *flexDirection* diset ke *row*, maka sumbu utama berada secara horizontal. Dalam kondisi ini, *justifyContent* dapat dimanfaatkan untuk mengatur posisi komponen anak secara mendatar, seperti memusatkan komponen di tengah kontainer.

Sebaliknya, ketika *flexDirection* diset ke *column*, sumbu utama berada secara vertikal. Pada pengaturan ini, *justifyContent* berperan untuk mengatur posisi komponen anak secara vertikal, misalnya memposisikan komponen tepat di tengah kontainer dari atas ke bawah.

`justifyContent: 'center'`

Properti value yang dapat digunakan:

Value	Deskripsi
flex-start (default)	Komponen anak ditempatkan di awal sumbu utama kontainer.
flex-end	Komponen anak ditempatkan di akhir sumbu utama kontainer.
center	Komponen anak ditempatkan di tengah sumbu utama kontainer.
space-between	Komponen anak disebar merata dengan jarak hanya di antara komponen, tanpa jarak di tepi kontainer.
space-around	Komponen anak disebar merata dengan jarak di sekitar setiap komponen, termasuk di awal dan akhir kontainer.
space-evenly	Komponen anak disusun dengan jarak yang sama persis di antara komponen dan tepi kontainer.

Tabel 4.2 List Value Properti pada *JustifyContent*

```
<View style={{ flex:1, flexDirection:"column", justifyContent:'center' }}>
  <View style={{ backgroundColor: "purple" }}>
    <Text>Box 1</Text>
  </View>
  <View style={{ backgroundColor: "green" }}>
    <Text>Box 2</Text>
  </View>
</View>
```



Gambar 4.4 Tampilan Objek ketika diberikan *JustifyContent*

4. Properti *alignItems*

Properti *alignItems* digunakan untuk mengatur posisi komponen anak (*children*) terhadap *cross axis* (sumbu silang) dari sebuah kontainer. Konsepnya sangat mirip dengan *justifyContent*, namun perbedaannya terletak pada sumbu yang digunakan.

Jika *justifyContent* bekerja pada *main axis* (sumbu utama), maka *alignItems* bekerja pada *cross axis*, yaitu sumbu yang tegak lurus terhadap arah *flexDirection*. Properti ini

sangat penting untuk mengatur perataan elemen secara vertikal atau horizontal, tergantung orientasi layout yang digunakan.

`alignItems:'stretch'`

Beberapa properti value yang dapat digunakan:

Value	Deskripsi
stretch	Komponen anak akan direntangkan agar memenuhi ukuran kontainer pada sumbu silang.
flex-start	Komponen anak disejajarkan ke awal sumbu silang kontainer.
flex-end	Komponen anak disejajarkan ke akhir sumbu silang kontainer.
center	Komponen anak ditempatkan di tengah sumbu silang kontainer.
baseline	Komponen anak disejajarkan berdasarkan garis dasar (<i>baseline</i>) teks yang sama.

Tabel 4.3 List Value Properti pada AlignItems

5. Properti alignContent

Properti alignContent digunakan untuk mengatur distribusi baris (lines) komponen di sepanjang cross axis (sumbu silang) pada sebuah kontainer. Properti ini hanya berpengaruh ketika komponen anak dibungkus ke lebih dari satu baris, yaitu saat flexWrap diset ke wrap.

Berbeda dengan alignItems yang mengatur posisi setiap item, alignContent berfokus pada pengaturan jarak dan perataan antar baris dalam sebuah layout flexbox.

Value	Deskripsi
flex-start	Baris-baris komponen disejajarkan ke awal sumbu silang kontainer.
flex-end	Baris-baris komponen disejajarkan ke akhir sumbu silang kontainer.
stretch	Baris-baris komponen direntangkan agar memenuhi ukuran kontainer pada sumbu silang.
center	Baris-baris komponen ditempatkan di tengah sumbu silang kontainer.
space-between	Ruang kosong dibagikan secara merata di antara baris-baris komponen.
space-around	Ruang dibagikan merata di sekitar baris, dengan jarak di tepi kontainer setengah dari jarak antar baris.
space-evenly	Seluruh jarak antar baris dan tepi kontainer dibagi sama besar.

Tabel 4.4 List Value Properti pada AlignContent

alignContent tidak akan berpengaruh jika:

- flexWrap masih bernilai nowrap
- Hanya terdapat satu baris komponen

Properti ini sangat berguna pada tampilan grid, galeri, atau daftar elemen yang banyak dan membungkus ke beberapa baris.

4.2.2 Styling menggunakan StyleSheet

StyleSheet merupakan modul bawaan React Native yang digunakan untuk mendefinisikan style terpisah dari logika komponen. Pendekatan ini membuat kode lebih:

- Terstruktur
- Mudah dibaca
- Mudah dipelihara

Penggunaan *StyleSheet* direkomendasikan karena memberikan optimasi performa dibandingkan penulisan style secara inline[8].

Styling pada React Native berfungsi untuk mengatur tampilan antarmuka aplikasi, seperti warna, ukuran, posisi, dan tata letak komponen. Berbeda dengan pengembangan web yang menggunakan CSS, React Native menggunakan pendekatan JavaScript Object Style, yang secara konsep mirip dengan CSS namun ditulis langsung dalam JavaScript.

Secara umum, terdapat tiga pendekatan utama dalam penerapan styling di React Native, yaitu inline styling, internal styling, dan eksternal styling.

1. Inline Styling

Inline styling adalah cara pemberian style secara langsung pada properti style di dalam komponen JSX.

Karakteristik:

- Ditulis langsung di dalam JSX
- Cocok untuk styling sederhana atau cepat
- Kurang direkomendasikan untuk proyek besar karena sulit dirawat

```
<View style={{ flex:1, flexDirection:"column", alignItems:'stretch' }}>
  <Text>Style inline</Text>
</View>
```

Kelebihan	Kekurangan
Praktis dan cepat, Mudah dipahami untuk pemula	Sulit dikelola jika style banyak dan tidak reusable

Tabel 4.5 Kelebihan dan Kekurangan Inline Styling

2. Internal Styling (StyleSheet)

Internal styling menggunakan *StyleSheet.create()* dari library react-native. Style didefinisikan di dalam file yang sama dengan komponen.

Karakteristik:

- Style terpisah dari JSX
- Lebih rapi dan terstruktur
- Paling sering digunakan dalam praktikum dan proyek nyata

```
export default function Latihan4() {
  return (
    <View style={styles.container}>
      <Text>Style Internal</Text>
    </View>
  );
}
```



```

}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "yellow"
  }
})

```

Kelebihan	Kekurangan
• Kode lebih bersih dan terorganisir • Mudah dibaca dan dipelihara • Mendukung optimasi performa	Masih terbatas pada satu file

Tabel 4.6 Kelebihan dan Kekurangan Internal Styling

3. Eksternal Styling (File Terpisah)

Eksternal styling adalah pengembangan dari internal styling, di mana style diletakkan pada file terpisah dan dapat digunakan kembali oleh banyak komponen.

Karakteristik:

- Cocok untuk proyek berskala menengah hingga besar
- Mendukung konsistensi tampilan
- Biasanya disimpan dalam folder styles atau themes

styles/global_style.js

```

import { StyleSheet } from 'react-native';

export const globalStyles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "yellow"
  }
});

```

Cara memanggil style external file global_style.js:

```

import React from "react";
import { View, Text, StyleSheet } from "react-native";
import { globalStyles } from '../styles/globalStyles';

export default function Latihan4() {
  return (
    <View style={styles.container}>
      <Text>Style External</Text>
    </View>
  );
}

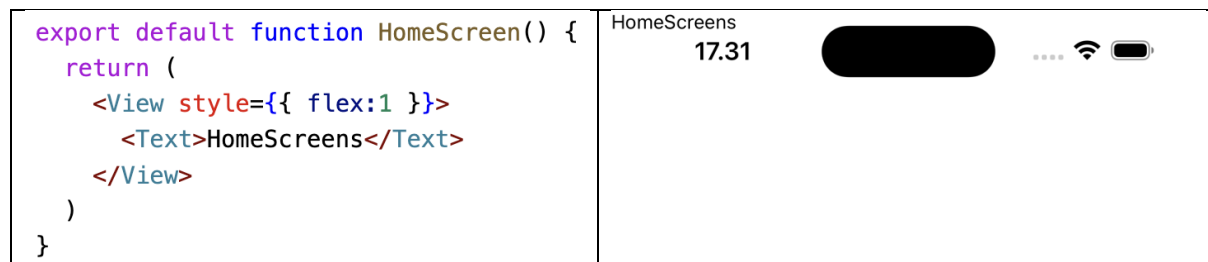
```

4.3 Praktikum

Dalam implementasinya, latihan ini memanfaatkan beberapa komponen kontainer seperti *View*, *ScrollView* dan lainnya untuk mengatur struktur layout, serta penggunaan objek dan *array of object* untuk merepresentasikan data buku, kategori, dan informasi lainnya secara terstruktur. Pendekatan Layout FlexBox dan penggunaan Styling Inline, Internal dan External. Pendekatan ini bertujuan untuk membiasakan penggunaan data dinamis dalam membangun antarmuka, sehingga tampilan tidak hanya bersifat statis, tetapi dapat dikembangkan lebih lanjut sesuai kebutuhan aplikasi nyata.

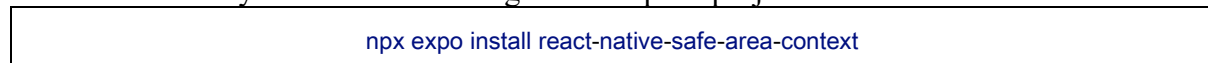
Langkah 1: Menambahkan library konten SafeAreaView sebagai konten container

Jika anda secara default menggunakan container pembungkus View pada awal komponen, maka posisi tampilan akan berada pada titik X = 0, seperti contoh dibawah ini:

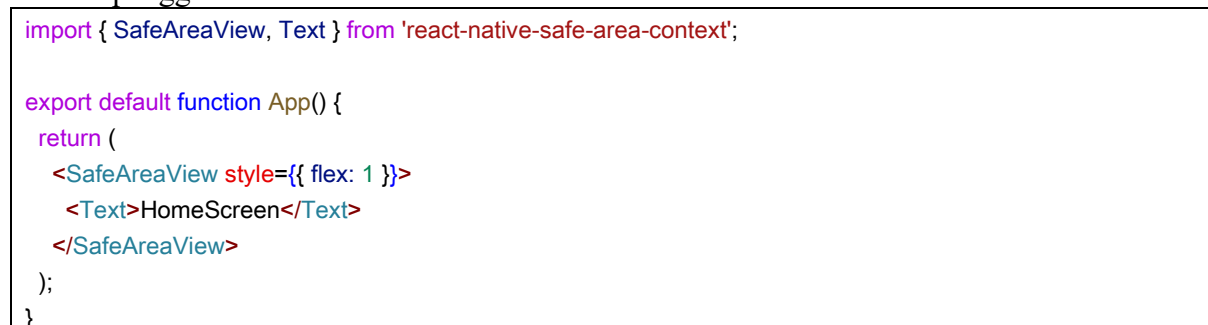


Untuk mengatasi hal tersebut ialah menggunakan library tambahan bernama SafeAreaView yang dimiliki oleh [react-native-safe-area-context](#). Komponen SafeAreaView berperan sebagai kontainer pembungkus (container) yang secara otomatis menyesuaikan padding berdasarkan safe area perangkat.

Tambahkan library SafeAreaView dengan sintax pada project anda:



Contoh penggunaan SafeAreaView:

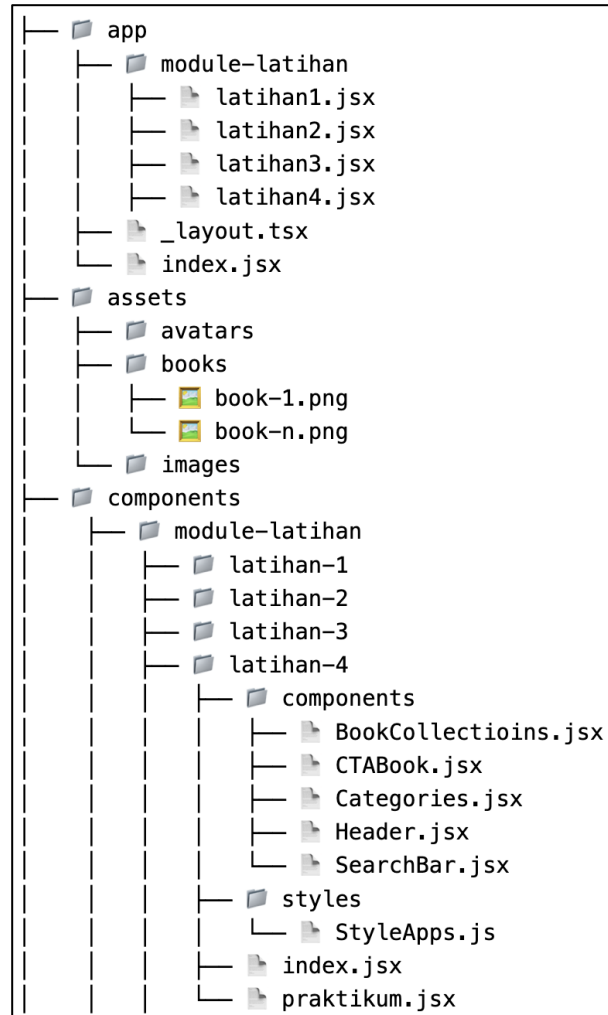


Library react-native-safe-area-context merupakan komponen penting dalam membangun antarmuka React Native yang aman, responsif, dan konsisten di berbagai perangkat. Dengan memanfaatkan SafeAreaView sebagai kontainer utama, tampilan aplikasi menjadi lebih profesional dan siap digunakan pada perangkat modern dengan berbagai variasi layar.

Jika diperlukan, saya dapat menambahkan contoh studi kasus penggunaan SafeAreaView pada layout form atau latihan praktikum khusus untuk bab Layout & Styling.

Langkah 2: Preparing project

Dalam project react-native ini, tambahkan beberapa folder dan file seperti yang ditampilkan pada gambar dibawah ini, dimana harus mempersiapkan beberapa file gambar dan simpan pada folder assets → images → books. Selain itu tambahkan folder components dan styles pada folder latihan-4.



Gambar 4.5 Struktur Folder Project

Setelah anda mempersiapkan beberapa file dan file JSX seperti susunan diatas maka selanjutnya ialah melanjutkan pada tahap latihan 3.

Dalam mengikuti kode pada latihan praktikum ini, perlu diperhatikan bahwa penulisan kode dilakukan secara berurutan sesuai dengan posisi dan urutan baris program. Setiap bagian kode, mulai dari proses import, pendefinisian data menggunakan objek dan array of object, hingga penyusunan komponen tampilan, saling berkaitan dan membentuk satu alur eksekusi yang utuh. Oleh karena itu, disarankan untuk tidak mengubah urutan baris kode agar proses pembelajaran dan pemahaman alur program dapat berjalan dengan baik.

Langkah 3: Membuat Styling eksternal dengan nama StyleApps.js

Pada latihan ini, pembahasan properti style tidak dijelaskan secara mendalam, karena fokus utama diarahkan pada pemahaman struktur komponen, pengelolaan data, dan penyusunan layout. Untuk mempelajari styling React Native lebih lanjut, silakan merujuk pada literatur

resmi dan sumber terpercaya seperti dokumentasi resmi React Native, Expo Documentation, serta referensi styling berbasis Flexbox yang tersedia pada React Native StyleSheet Documentation dan MDN Web Docs (Flexbox)[9].

Literatur terkait:

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties>

<https://reactnative.dev/docs/style>

<https://docs.expo.dev>

StyleApps.js

<pre> 1 import { StyleSheet } from "react-native"; 2 3 const color_list = { 4 orange: "#e8ab30", 5 green: "#49745e", 6 green_light: "#49745e35", 7 cream: "#f8f6f1", 8 white: "#fff", 9 }; </pre>	<pre> 11 const styles = StyleSheet.create({ 12 container: { 13 flex: 1, 14 backgroundColor: color_list.crem, 15 paddingHorizontal: 20 16 }, 17 title: { 18 fontWeight: "bold", 19 fontSize: 18, 20 color: "black" 21 }, </pre>
<pre> 22 sub_title: { fontSize: 16, color: "gray" }, 23 h_container: { 24 flexDirection: "row", 25 justifyContent: "space-between", 26 paddingVertical: 15, 27 }, 28 shadow: { 29 shadowColor: "black", 30 shadowOffset: { width: 0, height: 2 }, 31 shadowOpacity: 0.1, 32 shadowRadius: 3, 33 elevation: 3 34 }, 35 btn_icon: { 36 width: 45, 37 height: 45, 38 borderRadius: 22.5, 39 backgroundColor: color_list.white, 40 justifyContent: "center", 41 alignItems: "center", 42 }, </pre>	<pre> 43 new_com_container: { 44 borderRadius: 10, 45 backgroundColor: color_list.green, 46 padding: 20, 47 }, 48 new_book_img: { 49 width: 120, 50 height: 180, 51 borderRadius: 8, 52 }, 53 new_book_title: { 54 color: color_list.white, 55 fontSize: 18, 56 fontWeight: "bold", 57 }, 58 new_book_text: { 59 color: color_list.white, 60 fontSize: 13, 61 lineHeight: 16, 62 }, </pre>

<pre> 63 circle_premium: { 64 position: "absolute", 65 top: -5, 66 right: -5, 67 width: 30, 68 height: 30, 69 borderRadius: 15, 70 backgroundColor: color_list.orange, 71 justifyContent: "center", 72 alignItems: "center", 73 }, 74 btn_read: { 75 width: 150, 76 paddingVertical: 8, 77 backgroundColor: color_list.white, 78 borderRadius: 50, 79 justifyContent: "center", 80 alignItems: "center", 81 }, 82 btn_read_text: { 83 fontSize: 14, 84 color: color_list.green, 85 }, </pre>	<pre> 86 container_book_collections: { 87 flex: 1, 88 marginTop: 10, 89 }, 90 container_book_collections_title: { 91 fontSize: 18, 92 fontWeight: "bold", 93 }, 94 book_grid: { 95 flexDirection: "row", 96 flexWrap: "wrap", 97 justifyContent: "space-between", 98 marginTop: 10, 99 paddingBottom: 20, 100 }, 101 book_card: { 102 width: "48%", 103 backgroundColor: color_list.white, 104 borderRadius: 10, 105 marginBottom: 15, 106 overflow: "hidden", 107 }, </pre>
<pre> 108 book_card_img: { 109 width: "100%", 110 height: 200, 111 borderTopLeftRadius: 10, 112 borderTopRightRadius: 10, 113 }, 114 circle_premium_small: { 115 position: "absolute", 116 top: 5, 117 right: 5, 118 width: 24, 119 height: 24, 120 borderRadius: 12, 121 backgroundColor: color_list.orange, 122 justifyContent: "center", 123 alignItems: "center", 124 }, 125 book_card_title: { 126 fontSize: 14, 127 fontWeight: "bold", 128 color: "black", 129 marginBottom: 4, 130 minHeight: 34, 131 }, </pre>	<pre> 132 book_card_author: { 133 fontSize: 12, 134 color: "gray", 135 marginBottom: 8, 136 }, 137 book_card_footer: { 138 flexDirection: "row", 139 justifyContent: "space-between", 140 alignItems: "center", 141 }, 142 book_card_rating: { 143 fontSize: 12, 144 color: "black", 145 marginLeft: 4, 146 fontWeight: "600", 147 }, 148 book_card_views: { 149 fontSize: 12, 150 color: "gray", 151 marginLeft: 4, 152 }, 153 }); 154 155 export { styles, color_list }; </pre>

File ini berisi styling untuk aplikasi Book Discovery dengan struktur:

1. COLOR PALETTE (color_list):

- orange: #e8ab30 (untuk badge premium & rating)
- green: #49745e (warna utama/primary)
- green_light: #49745e35 (warna hijau transparan untuk badge)
- cream: #f8f6f1 (background utama aplikasi)
- white: #fff (background card & button)

2. LAYOUT UTAMA:

- container: Layout utama dengan background crem & padding horizontal 20
- h_container: Header container dengan flexDirection row & space-between

3. TYPOGRAPHY:

- title: Bold 18px untuk judul utama
- sub_title: 16px gray untuk subtitle
- new_book_title: Bold 18px white untuk judul buku featured
- new_book_text: 13px white untuk teks deskripsi
- book_card_title: Bold 14px untuk judul buku di card
- book_card_author: 12px gray untuk nama author

4. COMPONENTS:

a. HEADER SECTION:

- btn_icon: Button circular 45x45 dengan background putih
- shadow: Style untuk shadow effect (iOS & Android)

b. FEATURED BOOK (New Coming):

- new_com_container: Container hijau dengan border radius 10
- new_book_img: Image 120x180 dengan border radius 8
- circle_premium: Badge premium circular 30x30 orange
- btn_read: Button "Read Now" putih dengan border radius 50

c. BOOK COLLECTION GRID:

- book_grid: Grid layout dengan flexWrap & space-between
- book_card: Card 48% width dengan background putih & overflow hidden
- book_card_img: Image full width 200px height
- circle_premium_small: Badge premium kecil 24x24
- book_card_content: Padding 12px (implisit, perlu ditambahkan)
- book_card_footer: Footer dengan rating & views (flexDirection row)

5. FITUR KHUSUS:

- Shadow support untuk iOS (shadowColor, shadowOffset, shadowOpacity, shadowRadius)
- Shadow support untuk Android (elevation)
- Premium badge positioning (absolute positioning)
- Responsive grid layout (48% width untuk 2 kolom)
- Text truncation support (minHeight untuk konsistensi card)

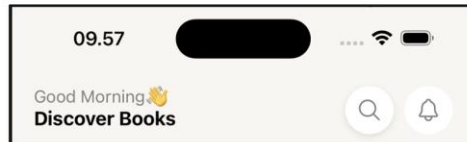
Langkah 4: Membuat RFC Header konten

Membuat tampilan header informasi berupa text pada file Header.jsx:

```
import Ionicons from "@expo/vector-icons/Ionicons";
import { Text, TouchableOpacity, View } from "react-native";
import { styles } from "../styles/StyleApps";

const Header = () => {
  return (
    <View style={styles.h_container}>
      <View>
        <Text style={styles.sub_title}>Good Morning👋</Text>
        <Text style={styles.title}>Discover Books</Text>
      </View>
      <View style={{ flexDirection: "row", gap: 10 }}>
        <TouchableOpacity style={styles.btn_icon, styles.shadow}>
          <Ionicons name="search-outline" size={24} color="gray" />
        </TouchableOpacity>
        <TouchableOpacity style={styles.btn_icon}>
          <Ionicons name="notifications-outline" size={24} color="gray" />
        </TouchableOpacity>
      </View>
    </View>
  );
};

export default Header;
```



Component header yang ditampilkan di bagian atas aplikasi (fixed, tidak ikut scroll)
KOMPONEN UTAMA:

1. Left Section (Greeting & Title):

- Sub Title: "Good Morning 👋" (gray, 16px)
- Main Title: "Discover Books" (bold, 18px)

2. Right Section (Action Buttons):

- Search Button: Icon search dengan shadow effect
- Notification Button: Icon notification tanpa shadow

FITUR:

- Menggunakan flexDirection: "row" dengan justifyContent: "space-between"
- Gap 10px antara kedua icon button
- Search button memiliki shadow effect (styles.shadow)
- Notification button tanpa shadow (plain)

DEPENDENCIES:

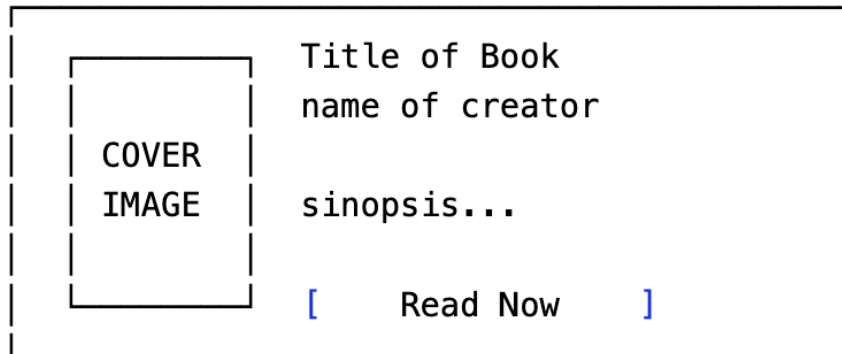
- React Native components (View, Text, TouchableOpacity)
- Ionicons untuk icon (search-outline & notifications-outline)
- StyleTugas4 untuk styling (h_container, sub_title, title, btn_icon, shadow)

Library atau dependencies Ionicons milik [@expo/vector-icons/Ionicons](https://icons.expo.fyi/) merupakan bawaan font icon dari react-native expo. Untuk mengetahui icon apa saja yang tersedia, silakan kunjungi <https://icons.expo.fyi/Index>

Langkah 5: Membuat RFC *Call to Action* (CTA) konten

Call to Action (CTA) adalah elemen pemasaran berupa teks, tombol, atau gambar yang dirancang untuk mendorong audiens melakukan tindakan spesifik segera. Dalam aplikasi ini CTA digunakan untuk menampilkan buku terakhir yang ditambahkan ke koleksi.

STRUKTUR VISUAL:



Komponen react dibawah ini berada pada file CTABook.jsx:



KOMPONEN BREAKDOWN:

1. CTABook (Main Component):

- Container utama dengan background hijau & shadow
- Layout horizontal (flexDirection: row)
- Menerima props: book object

2. CTAImage (Sub-component):

- Menampilkan cover buku (120x180px) dengan shadow
- Premium badge (crown icon) jika buku tidak gratis
- Badge positioned absolute di pojok kanan atas
- Props: book object

3. CTAInfoBook (Sub-component):

- Menampilkan informasi buku:
 - Title (bold, 18px, white)
 - Author (13px, white, prefix "by")
 - Sinopsis (13px, white, max 3 lines dengan ellipsis)
- Props: book object

4. ButtonRead (Sub-component):

- Button CTA "Read Now" dengan shadow
- Background putih, text hijau
- Border radius 50 (pill shape)
- Width 150px
- No props (static button)

PROPS STRUCTURE:

```
book = {  
  id: number,  
  title: string,  
  img: require('path'),  
  author: string,  
  is_free: boolean,  
  sinopsis: string,  
  rating: number,  
  views: number  
}
```

FITUR:

- Conditional rendering untuk premium badge (!book.is_free)
- Text truncation dengan numberOfLines & ellipsizeMode
- Multiple shadow effects (container, image, badge, button)
- Responsive layout dengan flex & flexShrink

STYLING:

- new_com_container: Green background container
- new_book_img: Book cover image style
- circle_premium: Premium badge (orange circle with crown)
- new_book_title: Book title style
- new_book_text: Author & sinopsis text style
- btn_read: Read button style
- shadow: Shadow effect untuk iOS & Android

DEPENDENCIES:

- React Native (View, Text, TouchableOpacity, Image)
- AntDesign icons (crown icon untuk premium badge)

- StyleTugas4 untuk styling

Library atau dependencies AntDesign milik [@expo/vector-icons/AntDesign](https://icons.expo.fyi/Index) merupakan bawaan font icon dari react-native expo. Untuk mengetahui icon apa saja yang tersedia, silakan kunjungi <https://icons.expo.fyi/Index>

Langkah 6: Membuat RFC kategori buku

File ini adalah komponen React Native yang menampilkan navigasi kategori dalam bentuk badge atau pill buttons. Menampilkan 4 kategori: "All", "Free", "Premium", "Popular" dan menggunakan `map()` untuk render setiap kategori sebagai tombol.

Komponen react dibawah ini berada pada file Categories.jsx:

```

1  import React from "react";
2  import { View, Text, TouchableOpacity, StyleSheet } from "react-native";
3  import { color_list } from "../styles/StyleApps";
4
5  const Categoriesnav = () => {
6    const categories = ["All", "Free", "Premium", "Popular"];
7    return (
8      <View style={styles.categ_badge_container}>
9        {categories.map((category, index) => (
10          <TouchableOpacity
11            key={index}
12            style={{
13              ...styles.categ_badge,
14              backgroundColor:
15                category === "All" ? color_list.green : color_list.green_light,
16            }}
17          >
18            <Text
19              style={{
20                color: category === "All" ? color_list.white : "dark",
21              }}
22            >
23              {category}
24            </Text>
25          </TouchableOpacity>
26        ))}
27      </View>
28    );
29  };
30
31  const styles = StyleSheet.create({
32    categ_badge_container: { flexDirection: "row", marginTop: 15 },
33    categ_badge: {
34      paddingHorizontal: 15,
35      paddingVertical: 10,
36      borderRadius: 20,
37      marginRight: 10,
38    },
39  });
40
41  export default Categoriesnav;

```



Styling Dinamis:

a. Kategori "All":

- Background hijau (`color_list.green`)
- Teks putih (`color_list.white`)

b. Kategori Lainnya:

- Background hijau muda (`color_list.green_light`)
- Teks gelap ("dark")

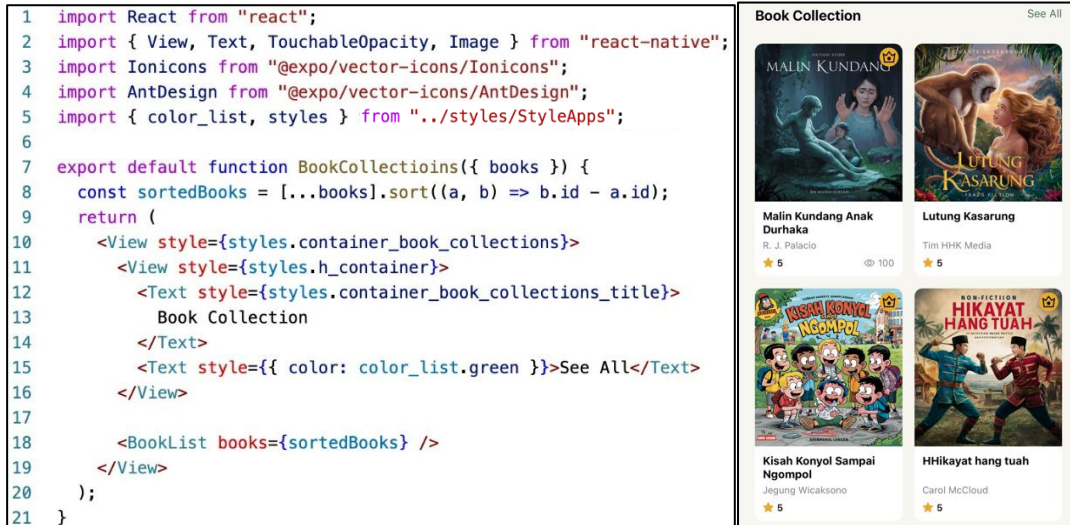
Struktur:

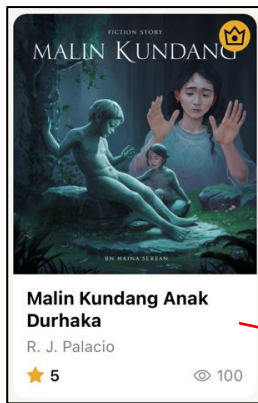
- Container horizontal (flexDirection: "row")
- Badge berbentuk pill (rounded corners dengan borderRadius: 20)
- Spacing antar badge dengan marginRight: 10

Komponen ini berfungsi sebagai filter visual untuk kategori buku dalam aplikasi, dengan kategori "All" sebagai pilihan default yang ter-highlight.

Langkah 7: Membuat RFC koleksi buku

File ini adalah komponen React Native yang menampilkan koleksi buku dalam bentuk grid layout. Komponen react dibawah ini berada pada file BookCollectionoins.jsx:





```

57 const BookItemContent = ({ book }) => {
58   return (
59     <View style={{ padding: 10 }}>
60       <Text
61         style={styles.book_card_title}
62         numberOfLines={2}
63         ellipsizeMode="tail"
64       >
65         {book.title}
66       </Text>
67       <Text style={styles.book_card_author} numberOfLines={1}>
68         {book.author}
69       </Text>
70       {/* RATING & VIEWERS */}
71       <View style={styles.book_card_footer}>
72         <View style={{ flexDirection: "row", alignItems: "center" }}>
73           <AntDesign name="star" size={14} color={color_list.orange} />
74           <Text style={styles.book_card_rating}>{book.rating}</Text>
75         </View>
76         {book.views && (
77           <View style={{ flexDirection: "row", alignItems: "center" }}>
78             <Ionicons name="eye-outline" size={14} color="gray" />
79             <Text style={styles.book_card_views}>{book.views}</Text>
80           </View>
81         )}
82       </View>
83       {/* END RATING & VIEWERS */}
84     </View>
85   );
86 };

```

Struktur Komponen:

1. BookCollections (Komponen Utama)

- Menerima props books (array data buku)
- Mengurutkan buku berdasarkan ID terbaru (b.id - a.id)
- Menampilkan header "Book Collection" dengan link "See All"
- Merender komponen BookList

2. BookList

- Menampilkan buku dalam grid 2 kolom (book_grid)
- Setiap buku adalah TouchableOpacity (dapat diklik)
- Menggunakan shadow effect untuk depth visual
- Merender 2 sub-komponen: BookItemImg dan BookItemContent

3. BookItemImg

- Menampilkan gambar cover buku
- Menampilkan badge crown premium (👑) jika is_free = false
- Badge diposisikan absolute di pojok kanan atas

4. BookItemContent

- Menampilkan informasi buku:
- Title (max 2 baris dengan ellipsis)
- Author (max 1 baris)
- Rating (★ + angka rating)
- Views (👁️ + jumlah views, jika ada)

Fitur Utama:

- Grid layout responsif (2 kolom)
- Sorting otomatis (buku terbaru di atas)
- Badge premium untuk buku berbayar
- Rating & views counter

- Text truncation dengan ellipsis
- Shadow effect untuk card

Langkah 8: Membuat RFC Main App dan data buku

File ini adalah screen utama (HomeScreen) aplikasi yang menampilkan katalog buku dengan berbagai komponen.

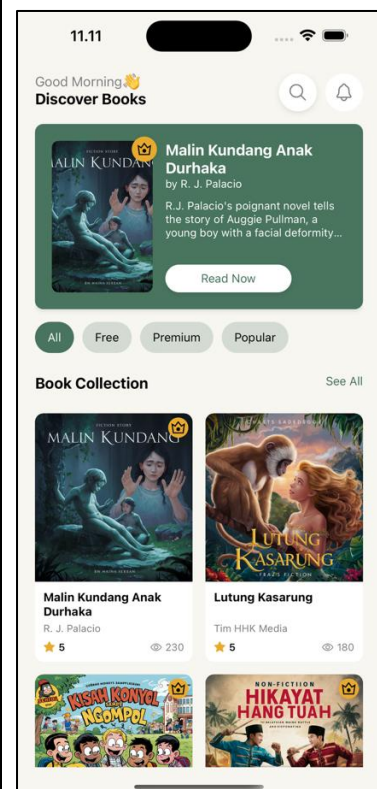
Layout Structure:

```
SafeAreaView (Container Utama)
├─ StatusBar (dark-content)
├─ Header (Good Morning + Search + Notification)
├─ ScrollView (Konten dapat di-scroll)
│   ├─ CTABook (Featured Book - buku terbaru)
│   ├─ Categoriesnav (All, Free, Premium, Popular)
│   └─ BookCollectionins (Grid katalog buku)
└─ Footer (Copyright text)
```

Gambar 4.6 Sturktur Container pada RFC

Komponen react dibawah ini berada pada file praktikum.jsx:

```
1  import { ScrollView, StatusBar, Text, View } from "react-native";
2  import { SafeAreaView } from "react-native-safe-area-context";
3  import { ListBook } from "../../constants/list_books";
4  import BookCollectionins from "../components/BookCollectionins";
5  import Categoriesnav from "../components/Categories";
6  import CTABook from "../components/CTABook";
7  import Header from "../components/Header";
8  import { color_list, styles } from "../styles/StyleApps";
9
10 export default function HomeScreen() {
11   const lastBook = ListBook[ListBook.length - 1];
12
13   return (
14     <SafeAreaView style={styles.container}>
15       <StatusBar barStyle={"dark-content"} />
16       { /* HEADER */ }
17       <Header />
18       { /* END HEADER */ }
19
20       <ScrollView
21         showsVerticalScrollIndicator={false}
22         contentContainerStyle={{ paddingBottom: 20 }}
23       >
24         { /* MAIN CONTENT */ }
25         <View style={{ flex: 1 }}>
26           <CTABook book={lastBook} />
27           <Categoriesnav />
28           <BookCollectionins books={ListBook} />
29         </View>
30         { /* MAIN CONTENT */ }
31
32         { /* FOOTER */ }
33         <View>
34           <Text style={{ color: color_list.green }}>
35             &copy; 2026 Febry Damatraseta Fairuz
36           </Text>
37         </View>
38         { /* END FOOTER */ }
39       </ScrollView>
40     </SafeAreaView>
41   );
42 }
43
```



Data Buku (ListBook):

Buatlah variable array object bernama ListBook pada `./constants/list_books.jsx`, dengan bentuk object dibawah ini sebagai bagian dari data item buku yang akan ditampilkan. Jumlah minimal item object sebanyak 2. Ikuti formati varibale dibawah ini:



Gambar 4.7 Struktur Folder Project dengan penambahan folder Data

list_books.jsx:

```
const ListBook = [
  {
    id: number (unique id),
    title: string,
    img: require('path directly to image file'),
    author: string,
    rating: number (1-5),
    views: number,
    is_free: boolean,
    sinopsis: string,
    story: string
  },
  ...
]
```

Struktur Komponen:

1. Import Dependencies

```
import { View, Text, StatusBar, ScrollView } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
```

- Komponen React Native dasar
- SafeAreaView untuk menghindari notch/status bar

2. Import Custom Components

```
import BookCollectionins from "../components/BookCollectionins";
import Categoriesnav from "../components/Categories";
import CTABook from "../components/CTABook";
import Header from "../components/Header";
```

Keempat komponen tersebut merupakan bagian dari MicroFE yang telah dibuat pada folder `components/*`.

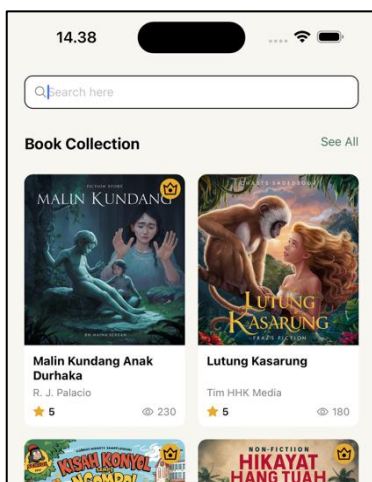
3. Data Management

```
const lastBook = ListBook[ListBook.length - 1];
```

- Mengambil buku terakhir dari array ListBook
- Digunakan untuk ditampilkan di komponen CTABook (featured book)

4.4 Tugas Latihan Praktikum

Pada tugas praktikum ini, pengembangan aplikasi dilanjutkan dengan menambahkan satu halaman baru yang difungsikan sebagai halaman pencarian buku. Halaman ini bertujuan untuk melatih pemahaman dalam pengelolaan komponen input, data berbasis array of object, serta interaksi pengguna melalui proses pencarian data secara dinamis. **Implementasi UI** untuk fitur pencarian ini merupakan salah satu fitur dasar yang umum ditemukan pada aplikasi nyata, khususnya aplikasi berbasis koleksi atau katalog.



Latihan 1: Membuat Halaman Search Page

Buatlah RFC untuk SearchPage dimana halaman ini akan menampilkan sebuah Text Input dan beberapa item buku yang telah dibuat pada latihan sebelumnya.

Output:

Tampilan akan seperti gambar disamping.

Latihan 2: Memanfaatkan reusable komponen

Pada gambar contoh disamping koleksi daftar item buku dapat anda panggil kembali tanpa perlu membuat komponen yang serupa.

Yaitu memanggil `<BookCollection books={ListBook} />` pada komponen ScreenPage.

Latihan 3: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 2, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project react expo yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository **"PPB-Praktikum-NPM"**, namun *commit* dengan nama **"Tugas Praktikum-4"**.
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).